

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	A Roja	Reg. No.:	192424226
Section / Batch:	2024 / Slot C	Date:	13-08-2026

Code of Conduct

I A Roja [192424226] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A Roja

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyse semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyse multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyse the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

Ans: Industry Problem

A banking company is developing an AI-powered customer support chatbot to understand and respond to customer queries. The chatbot currently uses Context-Free Grammar (CFG) for parsing. However, it faces problems such as ambiguity, subject–verb agreement, and inefficient parsing of long queries.

Given query:

“Show me the transactions with the card from last month”

(a) Analyse the Ambiguity Present in the Given Sentence Using CFG

The sentence has ambiguity because the phrases “with the card” and “from last month” can be attached to different parts of the sentence.

Interpretation 1 – Transactions made using the card

The customer wants to see transactions that were made using a card during the previous month.

Meaning:

Show transactions → using the card → from last month.

Semantic representation:

SHOW (transactions) AND METHOD (transactions, card) AND TIME (transactions, last_month)

Interpretation 2 – Transactions associated with the card

The phrase “with the card” can also mean transactions associated with a particular card.

Meaning:

Show the transactions associated with the card that occurred last month.

Simplified CFG Representation

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N$

PP → P NP

The sentence can be divided as:

Show | me | the transactions | with the card | from last month

V NP NP

The CFG can identify the grammatical structure, but it cannot always determine the intended meaning.

Ambiguity:

Phrase	Possible Interpretation
“With the card”	Card used for making transactions
“With the card”	Transactions associated with a particular card
“From last month”	Transactions occurred last month
“From last month”	May modify another part of the sentence depending on parsing

Therefore, CFG alone cannot reliably identify the customer's intended meaning.

(b) Evaluate the Limitations of CFG in Real-World Conversational Queries

CFG is useful for representing basic grammatical structures, but it has several limitations in banking chatbot applications.

1. Difficulty in handling ambiguity

A CFG can generate multiple valid parse trees for the same sentence.

Example:

“Show transactions with the card from last month.”

The system may not know whether “with the card” refers to the payment method or to a particular card.

2. Limited semantic understanding

CFG mainly describes syntax. It does not fully understand concepts such as:

- Transaction
- Card
- Account
- Payment method
- Date
- Transaction amount

Therefore, syntactic analysis alone is not enough for banking applications.

3. Subject–verb agreement

A basic CFG may not automatically detect agreement errors.

Example:

“The transactions is displayed.”

The correct sentence is:

“The transactions are displayed.”

Feature information is required to properly handle number and agreement.

4. Inefficient parsing of long queries

Long sentences may produce many possible parse trees. This increases:

- Parsing time
- Memory usage
- Computational cost

5. Difficulty handling conversational language

Users often provide short or incomplete queries such as:

“Transactions last month”

“Show card payments”

“What did I spend last month?”

A strict CFG may fail if these patterns are not included in the grammar.

6. No probability-based decision

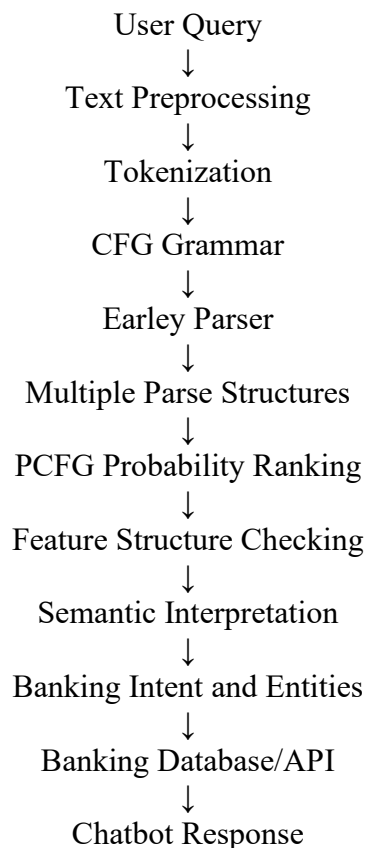
CFG determines whether a structure is grammatically possible, but it does not naturally determine which interpretation is more likely.

For a banking chatbot, the system needs to select the most probable interpretation.

(c) Propose an Improved Solution Using PCFG, Feature Structures, or Earley Parsing

A better solution is to combine CFG, PCFG, Feature Structures, and Earley Parsing.

Proposed Architecture



1. CFG

CFG can identify the basic syntactic structure of the query.

Example:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N$

$PP \rightarrow P NP$

CFG provides the basic grammatical framework.

2. Earley Parsing

Earley parsing is useful for handling general CFGs, ambiguous sentences, and incomplete input.

For the given query:

“Show me the transactions with the card from last month”

Earley parsing can identify different possible structures and retain alternative interpretations.

3. PCFG

Probabilistic Context-Free Grammar assigns probabilities to grammar rules.

Example:

$VP \rightarrow V NP PP \ 0.60$

$VP \rightarrow V NP PP \ 0.30$

$VP \rightarrow V NP \ 0.10$

When multiple interpretations are possible, PCFG can rank them and select the most probable interpretation.

Therefore, instead of only asking:

“Is this parse possible?”

the system can determine:

“Which valid interpretation is most probable?”

4. Feature Structures

Feature structures represent grammatical information such as:

Number = Singular

Person = Third

Tense = Present

For example:

transactions:

Number = Plural

are:

Number = Plural

This helps prevent errors such as:

“The transactions is displayed.”

and supports:

“The transactions are displayed.”

Semantic Representation

The system can extract:

Intent = SHOW_TRANSACTIONS

Object = Transactions

Payment Method = Card

Time = Last Month

Logical representation:

SHOW (transactions, payment_method = card, time = last_month)

This representation can be sent to the banking system to retrieve the required transactions.

(d) Justify How the Proposed Method Improves Accuracy and Efficiency in an Industry Setting

The combination of CFG, Earley parsing, PCFG, and Feature Structures provides a more reliable solution for a banking chatbot.

Requirement	CFG	Proposed Method
Basic syntax	Good	Excellent
Ambiguity handling	Limited	PCFG ranks alternatives
Incomplete queries	Poor	Earley handles partial parsing
Agreement checking	Limited	Feature Structures
Long queries	Can be inefficient	Earley provides efficient parsing
Probability-based interpretation	No	PCFG
Semantic extraction	Limited	Semantic processing layer
Conversational queries	Limited	Better handling
Industry scalability	Moderate	High

Accuracy Improvement

The proposed system improves accuracy because:

1. Earley Parser handles multiple possible structures.
2. PCFG selects the most probable interpretation.
3. Feature Structures handle subject–verb agreement.
4. Semantic analysis identifies banking-specific intents and entities.
5. Domain-specific banking rules can be added to the grammar.

For example, the query:

“Show me the transactions with the card from last month”

can be converted into:

Intent = SHOW_TRANSACTIONS

Payment Method = Card

Time Period = Last Month

The banking backend can then retrieve the relevant transaction records.

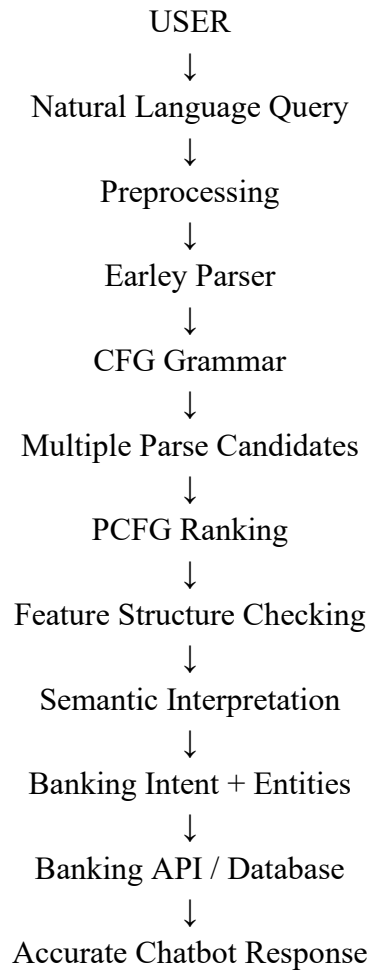
Efficiency Improvement

Earley parsing stores intermediate parsing results and avoids unnecessary repeated parsing. This makes it useful when a sentence has multiple possible structures.

PCFG ranks the possible interpretations and helps the system select the most probable one.

Therefore, the chatbot can provide faster and more reliable responses.

Industry-Level Architecture



Conclusion

A basic CFG is useful for identifying grammatical structures, but it is not sufficient for a real-world banking chatbot because conversational queries contain ambiguity, incomplete expressions, agreement requirements, and domain-specific meanings.

A combination of CFG, Earley Parsing, PCFG, and Feature Structures provides a stronger solution. Earley Parsing efficiently handles alternative and partial parses, PCFG resolves ambiguity by selecting the most probable interpretation, and Feature Structures improve grammatical accuracy.

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

a) Analyse how the given sentence can lead to multiple parse structures

b) Analyse the limitations of top-down parsing in this real-time application

c) Analyse how Earley parsing can improve handling of ambiguity and partial inputs

d) Compare and analyse the performance of top-down vs Earley parsing for such industry use cases

Ans: Industry Problem

A voice assistant system such as Alexa or Siri processes spoken commands using natural language parsing. The current system uses top-down parsing, but it faces problems such as backtracking, difficulty handling incomplete input, ambiguity, and slow real-time response.

Given command:

“Book a flight to Delhi with a window seat”

(a) Analyse How the Given Sentence Can Lead to Multiple Parse Structures

The sentence contains the phrase “with a window seat”, which can create attachment ambiguity.

Interpretation 1 – Window seat is requested for the flight

The user wants to book a flight to Delhi and wants a window seat.

Meaning:

BOOK (flight) AND DESTINATION (flight, Delhi) AND SEAT (flight, window)

Interpretation 2 – “With a window seat” is associated with the booking

The phrase can be treated as an additional condition or property of the booking.

Meaning:

BOOK (flight, destination = Delhi, seat = window)

For a voice assistant, both structures may initially appear grammatically possible.

Simplified Parse Structure

Sentence:

“Book a flight to Delhi with a window seat”

Possible structure:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N$

$PP \rightarrow P NP$

The sentence can be divided as:

Book | a flight | to Delhi | with a window seat

V NP PP

The phrase “to Delhi” specifies the destination, while “with a window seat” specifies the user's seating preference.

Semantic Representation

The intended meaning can be represented as:

BOOK (

flight,

destination = Delhi,

seat preference = window

)

The main challenge is correctly identifying the relationship between the phrases and the booking action.

(b) Analyse the Limitations of Top-Down Parsing in a Real-Time Application

Top-down parsing starts from the start symbol and attempts to generate the input sentence by applying grammar rules.

Although it is simple to understand, it has several limitations in voice assistant applications.

1. Backtracking

If the parser chooses an incorrect grammar rule, it may need to go back and try another rule.

For example:

“Book a flight to Delhi with a window seat”

If the parser initially attaches “with a window seat” incorrectly, it may have to backtrack and try another interpretation.

This increases processing time.

2. Difficulty with Ambiguous Input

Spoken language often contains ambiguous expressions.

For example:

“Book a flight to Delhi with a window seat.”

The parser must determine whether “with a window seat” is a seat preference or another part of the flight description. Top-down parsing may explore several possibilities.

3. Incomplete Input

Voice assistants frequently receive incomplete commands.

Examples:

“Book a flight to Delhi...”

“Book a flight...”

“Delhi tomorrow...”

A traditional top-down parser may not handle incomplete input efficiently.

4. Real-Time Processing

Voice assistants need to respond quickly.

Repeated backtracking and exploring unnecessary grammar rules can increase latency.

This can result in:

- Slow responses
- Poor user experience
- Increased computational cost

5. Speech Recognition Errors

Voice assistants receive input from speech recognition systems.

The recognized text may contain errors such as:

“Book a flight to Delhi with a window seat.”

being recognized incorrectly.

A rigid top-down parser may have difficulty handling such noisy input.

6. Limited Partial Parsing

Users may stop speaking temporarily or change their command while speaking.

A real-time voice assistant needs to interpret partial input as it arrives.

Traditional top-down parsing is less suitable for this requirement.

(c) Analyse How Earley Parsing Improves Handling of Ambiguity and Partial Inputs

Earley parsing is a chart-based parsing algorithm that can efficiently handle general context-free grammars.

It is particularly useful for voice assistants because it can handle:

- Ambiguous sentences
- Incomplete sentences
- Different grammatical structures
- Partial input
- Left-recursive grammar rules

Earley Parser Processing

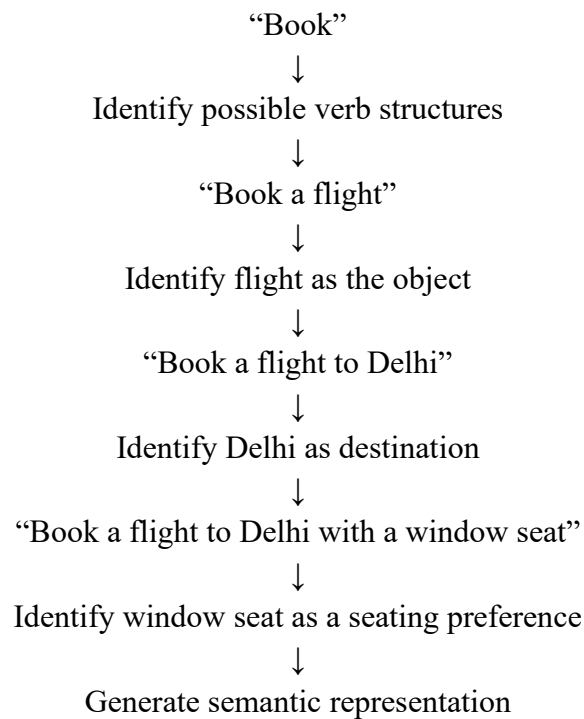
For the command:

“Book a flight to Delhi with a window seat”

the parser processes the input incrementally.

Simplified process:

Input:



Semantic Output

The parser can produce:

Intent = BOOK_FLIGHT

Destination = Delhi

Seat Preference = Window

This can then be passed to the flight booking system.

Handling Partial Input

Suppose the user says:

“Book a flight to...”

The Earley parser can maintain the current parsing state while waiting for additional input.

If the user continues:

“Delhi...”

the parser can update the existing structure.

Final input:

“Book a flight to Delhi with a window seat.”

The parser completes the interpretation.

This is useful for real-time voice assistants.

(d) Compare and Analyse Top-Down Parsing vs Earley Parsing

Feature	Top-Down Parsing	Earley Parsing
Parsing direction	Starts from start symbol	Processes input incrementally
Ambiguity	Difficult to manage	Handles ambiguity effectively
Backtracking	Often required	Reduced through chart parsing

Feature	Top-Down Parsing	Earley Parsing
Partial input	Limited	Good support
Incomplete sentences	Less effective	More effective
Real-time applications	Less suitable	More suitable
Left recursion	Can cause problems	Can handle it
General CFG support	Depends on implementation	Supports general CFG
Memory usage	Can be lower for simple grammar	Uses chart/state storage
Passing efficiency	Good for simple grammar	Good for general CFG
Voice assistant suitability	Moderate	High

Top-Down Parsing

Advantages:

- Simple to implement
- Easy to understand
- Works well for simple and predictable grammar
- Can be efficient for small grammar systems

Disadvantages:

- Backtracking can increase processing time
- Poor handling of ambiguity
- Difficult to handle incomplete input
- Less suitable for complex conversational commands

Earley Parsing

Advantages:

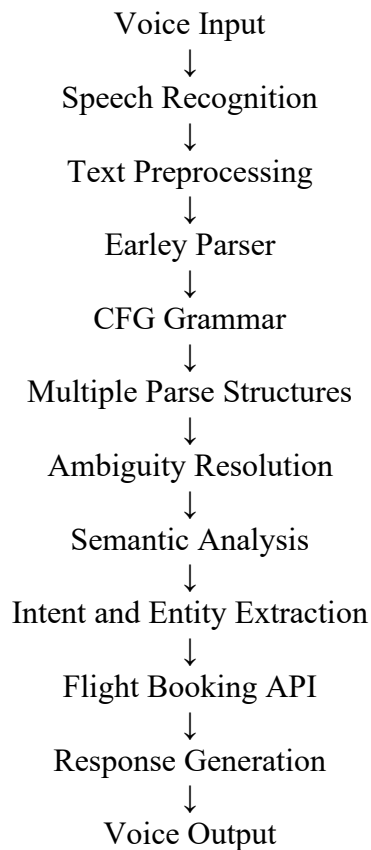
- Handles ambiguous grammar
- Supports general CFG
- Handles partial and incomplete input
- Reduces unnecessary repeated parsing through chart states
- Suitable for complex language structures

Disadvantages:

- More complex to implement
- Requires additional memory for chart/state storage
- May be unnecessary for very simple grammars

Proposed Voice Assistant Architecture

The improved voice assistant can use the following architecture:



For the given command:

“Book a flight to Delhi with a window seat”

the structured output becomes:

Intent: BOOK_FLIGHT

Destination: Delhi

Seat Preference: Window

Industry Justification

Earley parsing is more suitable for a real-time voice assistant because voice commands are often:

- Ambiguous
- Incomplete
- Long
- Conversational
- Continuously received
- Affected by speech recognition errors

The Earley parser can maintain multiple possible interpretations without immediately discarding them.

Once sufficient input is received, semantic analysis can select the appropriate interpretation.

For example:

User:

“Book a flight to Delhi...”

System:

Recognizes that the user intends to book a flight and waits for additional information.

User:

“...with a window seat.”

System:

Adds the seat preference.

Final structured representation:

BOOK_FLIGHT

Destination = Delhi

Seat = Window

This reduces unnecessary backtracking and supports faster real-time processing.

Conclusion

Top-down parsing is simple and useful for small grammar systems, but it becomes less effective when used in real-time voice assistants because of backtracking, ambiguity, and incomplete input.

Earley parsing provides a better solution because it supports general CFGs, handles ambiguity, processes input incrementally, and can work with partial sentences.

Therefore, for an industry-level voice assistant, Earley parsing combined with semantic analysis and intent/entity extraction is more suitable than a traditional top-down parsing approach.

3. A healthcare company is developing an AI system to analyse patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

Ans: Industry Problem

A healthcare company is developing an AI-based NLP system to analyse patient medical reports. The system must automatically extract important information such as diagnosis, prescribed treatments, follow-up actions, and locations.

The system must handle:

- Complex and lengthy medical sentences
- Ambiguous sentence structures
- Subject–verb agreement
- Relationships between medical actions and entities
- Sub-categorization of medical verbs
- Real-time processing
- Large-scale hospital data

Given Input

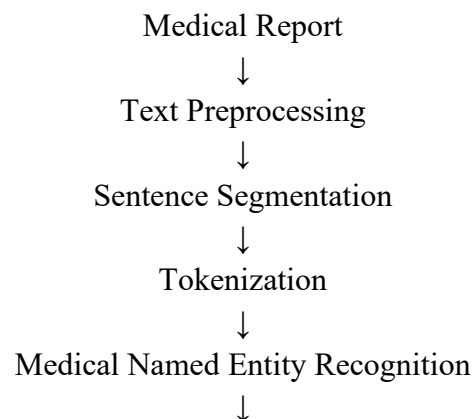
“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

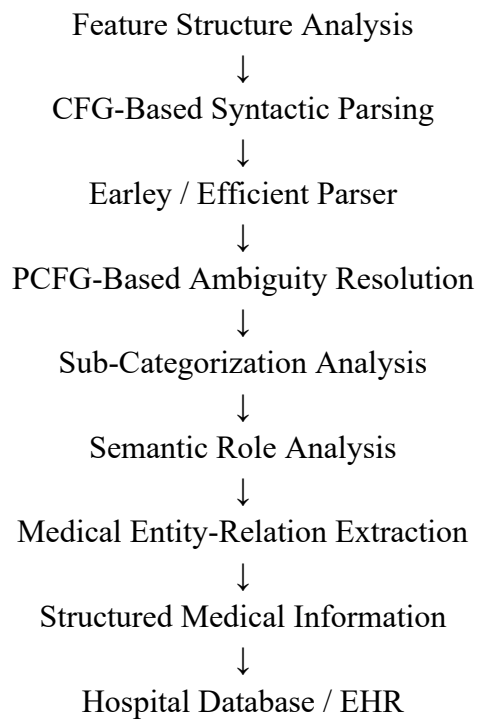
The existing system uses only basic CFG and therefore may fail to correctly identify relationships between the doctor, patient, medication, follow-up action, and location.

(a) Design a Complete NLP Architecture

A hybrid NLP architecture using CFG, PCFG, Feature Structures, and efficient parsing techniques can provide better performance.

Proposed Architecture





1. Text Preprocessing

The input medical report is first cleaned and divided into sentences.

Tasks include:

- Sentence segmentation
- Tokenization
- Removal of unnecessary symbols
- Normalization of text
- Medical abbreviation handling

Example:

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

is divided into meaningful tokens:

The | doctor | who | reviewed | the | patient | last | week | recommends | starting | medication | and | scheduling | a | follow-up | visit | in | Chennai

2. Named Entity Recognition

The system identifies important medical entities.

For example:

Entity	Type
doctor	Medical Professional
patient	Patient
medication	Treatment
follow-up visit	Medical Action
Chennai	Location

Entity	Type
last week	Time

This helps the system understand what each word or phrase represents.

3. CFG for Syntactic Structure

CFG is used to identify the basic grammatical structure.

Example rules:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$NP \rightarrow NP REL Clause$

$VP \rightarrow V NP$

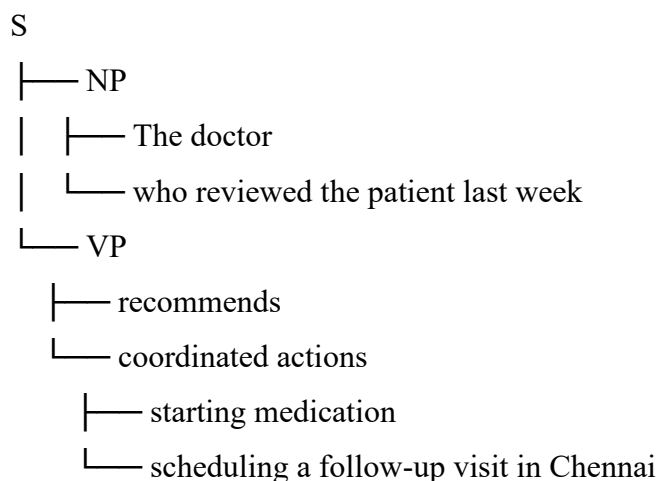
$VP \rightarrow V Gerund Phrase$

$VP \rightarrow VP Conj VP$

$REL Clause \rightarrow REL Pronoun VP$

$PP \rightarrow Prep NP$

A simplified structure is:



CFG provides the syntactic structure, but it cannot completely determine the meaning.

4. PCFG for Ambiguity Resolution

Medical sentences can have multiple possible interpretations.

A PCFG assigns probabilities to different grammar rules and helps select the most probable parse.

For example:

$VP \rightarrow V Gerund Phrase$ 0.50

$VP \rightarrow V NP$ 0.30

$VP \rightarrow VP Conj VP$ 0.20

If several parse structures are possible, the system selects the structure with the highest probability.

This improves the interpretation of complex medical sentences.

5. Feature Structures for Agreement

Feature structures represent grammatical properties such as:

- Number
- Person
- Tense
- Gender where relevant

For example:

doctor:

Number = Singular

Person = Third

recommends:

Number = Singular

Person = Third

Tense = Present

Therefore:

“The doctor recommends...”

is accepted.

But:

“The doctor recommends...”

is identified as an agreement error.

Feature structures improve syntactic accuracy.

6. Efficient Parsing

For long medical reports, an efficient parser such as the Earley parser can be used.

It can handle:

- Complex CFGs
- Ambiguous structures
- Long sentences
- Partial input
- Multiple possible parses

This is useful in hospitals where medical information must be processed quickly.

(b) Step-by-Step Workflow for the Given Sentence

Input

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

Step 1 – Input Acquisition

The system receives the medical report as text.

The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.

Step 2 – Sentence Segmentation

The system identifies one complete sentence.

Sentence 1:

The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.

Step 3 – Tokenization

The sentence is divided into tokens.

The
doctor
who
reviewed
the
patient
last
week
recommends
starting
medication
and
scheduling
a
follow-up
visit
in
Chennai

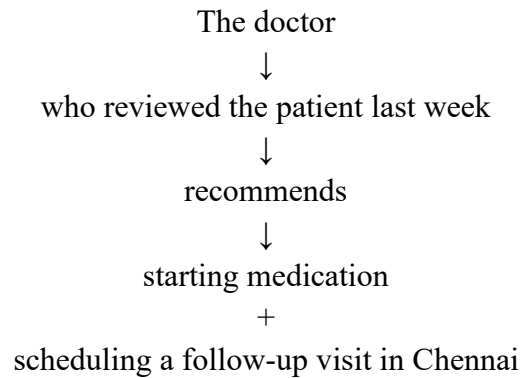
Step 4 – Entity Identification

The system identifies relevant entities.

Text	Entity Type
doctor	Medical professional
patient	Patient
last week	Time
medication	Treatment
follow-up visit	Medical action
Chennai	Location

Step 5 – Syntactic Parsing

The parser identifies the grammatical structure.



The relative clause:

“Who reviewed the patient last week”

describes the doctor.

Step 6 – Feature Structure Analysis

The system checks agreement.

Subject:

doctor

Number = Singular

Verb:

recommends

Number = Singular

Tense = Present

The agreement is correct:

“The doctor recommends...”

Step 7 – PCFG-Based Ambiguity Resolution

The system generates possible parse structures and assigns probabilities.

The most probable interpretation is:

- The doctor reviewed the patient.
- The doctor recommends starting medication.
- The doctor recommends scheduling a follow-up visit.
- The follow-up visit is in Chennai.

Step 8 – Semantic Analysis

The system identifies the relationships between entities and actions.

Logical representation:

REVIEW (doctor, patient, last week)

RECOMMEND (doctor, start(medication))

RECOMMEND (doctor, schedule(follow_up_visit))

LOCATION (follow_up_visit, Chennai)

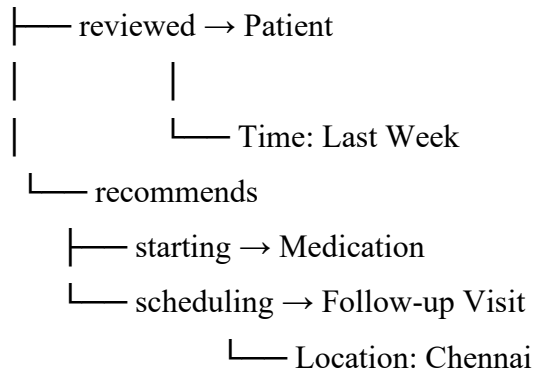
Step 9 – Structured Output

The final structured information can be represented as:

Category	Extracted Information
Medical Professional	Doctor
Patient	Patient
Previous Action	Reviewed patient
Time	Last week
Recommended Treatment	Start medication
Follow-up Action	Schedule follow-up visit
Location	Chennai

Final Semantic Representation

Doctor



(c) Method for Sub-Categorization, Real-Time Processing, Accuracy and Scalability

1. Incorporating Sub-Categorization Frames for Medical Verbs

Sub-categorization describes what types of arguments a verb requires.

Medical verbs have different patterns.

Example 1 – Review

Verb:

“review”

Frame:

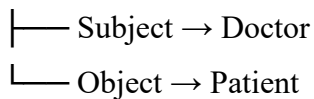
Review (DOCTOR, PATIENT)

Example:

“The doctor reviewed the patient.”

Structure:

review



Example 2 – Recommend

Verb:

“recommend”

Frame:

Recommend (DOCTOR, ACTION)

Example:

“The doctor recommends starting medication.”

Structure:

recommend

|—— Subject → Doctor

|—— Action → Starting medication

Example 3 – Start

Verb:

“start”

Frame:

Start (PATIENT, MEDICATION)

The system can identify that medication is the object associated with the action.

Example 4 – Schedule

Verb:

“schedule”

Frame:

Schedule (PERSON, VISIT)

Example:

“Scheduling a follow-up visit.”

Sub-Categorization Frame Table

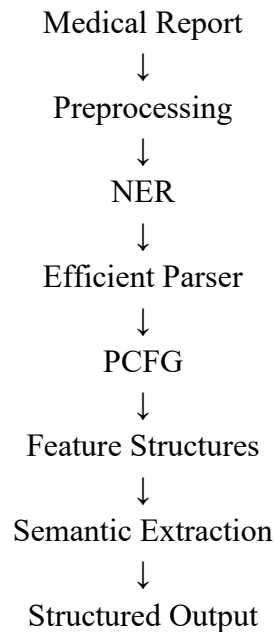
Medical Verb	Subject	Object / Complement	Example
review	Doctor	Patient	Doctor reviewed patient
diagnose	Doctor	Disease/Condition	Doctor diagnosed diabetes
prescribe	Doctor	Medication	Doctor prescribed medicine
recommend	Doctor	Action/Treatment	Doctor recommends treatment
start	Patient/Doctor	Medication/Treatment	Start medication
schedule	Staff/Doctor	Appointment/Visit	Schedule follow-up visit
monitor	Doctor	Patient/Condition	Monitor patient

These frames help the NLP system correctly identify relationships.

2. Real-Time Data Processing

Hospitals require fast processing of medical reports.

The proposed system can use a pipeline-based architecture:



Methods for improving real-time processing

1. Use efficient chart parsing.
2. Process sentences independently when possible.
3. Cache frequently used grammar rules.
4. Use optimized medical dictionaries.
5. Use parallel processing for multiple reports.
6. Store extracted information directly in structured format.
7. Use incremental parsing for incoming reports.

3. Improving Accuracy

Accuracy can be improved by combining multiple NLP techniques.

CFG

Provides syntactic structure.

PCFG

Resolves ambiguous structures using probabilities.

Feature Structures

Handle grammatical agreement.

Named Entity Recognition

Identifies medical entities.

Sub-Categorization Frames

Identify relationships between verbs and their arguments.

Medical Vocabulary

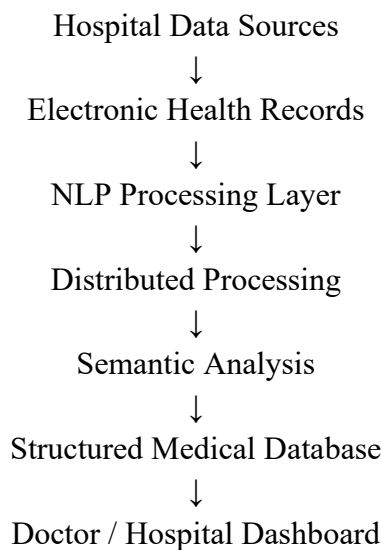
A domain-specific medical dictionary improves recognition of:

- Diseases
- Medicines
- Treatments
- Procedures
- Body parts
- Medical professionals

4. Improving Scalability in a Hospital Environment

The system should be designed to process large amounts of medical data.

Scalable Architecture



The system can process multiple medical reports simultaneously.

For example:

Report 1 → NLP Engine → Structured Data

Report 2 → NLP Engine → Structured Data

Report 3 → NLP Engine → Structured Data

Report 4 → NLP Engine → Structured Data

This makes the system suitable for large hospitals.

Overall Proposed Architecture



Expected Structured Output

For the given sentence:

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

The system should produce:

Doctor:

Entity = Doctor

Patient:

Entity = Patient

Previous Action:

Review (Doctor, Patient)

Time:

Last Week

Treatment:

Start (Medication)

Recommendation:

Doctor recommends starting medication

Follow-up:

Schedule (Follow-up Visit)

Location:

Chennai

Conclusion

A basic CFG is not sufficient for analysing complex healthcare reports because medical sentences contain ambiguity, long structures, grammatical relationships, and domain-specific terminology.

A hybrid NLP architecture using CFG, PCFG, Feature Structures, Earley Parsing, Named Entity Recognition, and Sub-Categorization Frames provides a more accurate solution.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____